

三种参考方法的详细分析

795、Bowman/hybrid 与 Lin2025 在当前项目中的作用、差异与改进空间

2026-04-17

1 文档目的

本文档面向项目汇报与答疑场景，对当前系统所参考的三种主要方法进行单独分析。三种方法分别为：

1. 原始 795 经验参考路线；
2. Bowman 风格的物理优化路线，在当前代码中主要对应 `hybrid_only`；
3. Lin2025 风格的学习型初始化路线，在当前代码中主要对应 `lin2025_plus_hybrid`。

本文重点回答以下问题：

1. 三种方法各自解决什么问题；
2. 三种方法在当前工程中如何衔接；
3. 当前方法相对于 795 和 Bowman 的提升体现在什么位置；
4. 当前方法仍然存在哪些局限；
5. 进一步优化时最值得投入的方向是什么。

2 AutoDL notebook 版本演进与当前主线

当前汇报包中关于 AutoDL notebook 的说明，不再只依据单个当前附件，而是回读了 `colab_local_split/` 下从 v2 到 v58 的整条版本链。重建方法同时参考了每版 notebook 的标题、首段说明、内置 `Change Log/Key changes`、关键单元结构以及 v37-v53 自带的历史表，因此可以把“算法主拐点”“结构修复”“文案漂移”三类变化区分开。

需要特别说明三件事。第一，v2-v24 中有一批 notebook 的首段中文说明已经出现乱码，不能只靠标题逐字摘抄，需要结合后续历史表回推。第二，v20、v30、v50-v52、v54-v55、v57 都存在标题或首段说明继承旧版本的情况。第三，v45-v49 的历史表虽然已经出现，但

表头和“当前版本如何处理”一栏仍明显滞后，因此版本史必须人工重建，而不能机械复制。

表 1: AutoDL notebook 从 v2 到 v58 的版本演进重建表

版本	阶段定位	主要改动/实验目标	暴露的问题或学到的结论	对当前主线的影响
阶段一：v2-v10，最早的 AutoDL 自包含化搭建期				
v2	自包含起点	首次把写源码、训练、导出、benchmark、自动分析合并到单 notebook	证明远程单文件工作流可行	作为后续全部版本的骨架起点
v3	起步迭代	延续 v2 结构，围绕默认流程与参数做小步修补	说明单文件流程可以稳定复用	保留为流程连续性样本
v4	起步迭代	继续补齐远程导出与本地分析衔接	远程训练不能脱离本地统一分析	形成后续 bundle 导出思路
v5	起步迭代	稳定依赖检查、导出目录和 benchmark 调用顺序	自包含 notebook 模式可持续维护	保留为结构延续版本
v6	早期 flat-top 调参	开始围绕 legacy flat-top 任务做训练/benchmark 试跑	当时任务定义仍未稳定	只保留为历史过渡
v7	早期 flat-top 调参	沿 v6 继续收敛 notebook 默认流程	仅有流程级改良，算法主线未定型	作为早期试跑记录
v8	早期 benchmark 成形	开始尝试把不同路线放到同一 notebook 中对比	统一 benchmark 是后续必要能力	为 v23 奠定结构前提
v9	稳定性修复	参考相位生成改走 CPU，规避 AutoDL 上 FFT / cuFFT 报错	早期最大问题是可运行性而非算法强弱	CPU 生成参考相位的思路被后续保留
v10	阶段收束	把早期 19-cell 单文件骨架基本固定下来	结构已经足以支持后续大幅扫描	作为 v11-v18 的入口
阶段二：v11-v18，早期 legacy task / flat-top 调参与训练期				
v11	显存策略拐点	训练回退策略改成“先降 batch，不先降模型容量”	显存问题不能简单通过缩模型解决	这条 OOM 处理思路被历史表明确保留
v12	训练稳定化	延续 v11，继续收紧显存与 teacher 配置	训练主循环开始比结构重写更重要	作为 v11 的延长验证
v13	训练稳定化	继续在 legacy task 下试图稳定训练与导出链	说明当时任务定义仍未与 line-flat 完全对齐	只保留为过渡版本
v14	目标定义对齐	flat-top 参考统一为带 sx/sy 的 line-flat 形式	任务定义必须先与实验目标对齐	是 line-flat 主线的第一个关键前置点
v15	teacher 池收紧	teacher 池更明确收紧到 line-flat 任务，并提高 hybrid 角色	lin2025 不能脱离 teacher 质量谈表现	保留为 line-flat teacher 清理起点
v16	蒸馏强化	继续加强 phase imitation 与 line-flat teacher 清理	learned init 开始有帮助，但仍不稳	作为 v19 之前的过渡增强
v17	蒸馏强化	延续 v16，继续改善 line-flat 训练主线	说明单纯加 epoch 不是主要方向	保留为训练演化记录
v18	checkpoint 预筛选	在 v19 之前做早期 checkpoint 与 route 收缩	需要先筛 teacher / checkpoint，再谈最终路线	为 v19 的首次较好结果铺路
阶段三：v19-v30，line-flat 主线形成期				
v19	早期可用主线	line-flat 指标首次出现较像样的 core metrics	证明 learned init + hybrid 的方向是对的，但还不够稳	被后续历史表保留为过渡锚点
v20	中期过渡版	overlap / uniformity 已不错，但 core phase 仍偏弱；文件标题误写成 v21	数值有参考价值，但不适合作主骨架	只保留为历史中间态

续下页

版本	阶段定位	主要改动/实验目标	暴露的问题或学到的结论	对当前主线的影响
v21	过渡延续版	延续 v20 家族并继续比较主路线	说明当时主线仍在快速摆动	未进入当前主线锚点
v22	过渡延续版	继续统一 benchmark 与导出分析链路	notebook 更像实验台，而非稳定交付件	只作过渡记录
v23	benchmark 成形点	首次把 795_only、795_plus_hybrid、balanced、quality、feedback_proxy 放进同一套 benchmark	终于能在同一环境横向比较所有主路线	这一结构一直保留到后续版本
v24	报告收束版	沿 v23 做结果输出和分析文案整理	统一 benchmark 的可读性开始重要起来	为 v25 结果查看改进铺路
v25	结果查看拐点	结果查看不再截断长 JSON，且高分辨率 1024x1272 导出链更明确	需要在 notebook 内直接读到关键信息，而不是只看一小段输出	结果展示方式被后续沿用
v26	主线延续版	延续 line-flat + 高分辨率导出主线，不再扩家族	说明需要先稳住主结构	作为 v25-v29 连续验证
v27	主线延续版	继续收紧 line-flat 训练和导出 workflow	主线已可复跑，但还缺清晰排序逻辑	保留为连续试跑版本
v28	排序逻辑转向	主排序切到 core-phase-first，并把 feedback_proxy 明确标成 exploratory route	BEST ROUTE 开始更贴近真实目标优先级	成为后续 benchmark 排名准则基础
v29	quality 收窄版	在 core-phase-first 前提下进一步收窄 quality sweep	说明需要围绕 winning family 局部细扫，而不是反复扩家族	为 v30 铺路
v30	第一主拐点	严格围绕 core_phase_flatness -> core_uniformity_loss -> overlap_loss 设计 baseline / phase_first / phase_lockin 质量家族；文件标题仍写成 v29	这是最强、最可复用的 learned 主线来源	当前版本史中第一个算法主锚点
阶段四：v31-v39，主线稳定前的过渡扫参与整合期				
v31	主线回挂版	标题仍沿用 v30，并开始围绕 v30 主线做小范围新试验	元数据开始出现继承漂移，不能只看标题判断内容	只作为过渡承接
v32	过渡清理版	延续 v31，继续做 post-v30 局部收敛	说明主线虽强，但仍缺因果拆解	不作为关键锚点
v33	因果对照版	在 quality 主线内定向对照 checkpoint / warm start / post-correction / reference bias / soft flux guard	低效率 basin 需要拆成更细的因果因素来看	为 v34-v35 的窄扫参提供方向
v34	低效率诊断版	收窄到 best_supervised + flux_guard 家族，显式诊断低效率 basin	说明部分质量改良是以效率坍塌为代价的	为后续回退提供证据
v35	极窄质量主线	把质量主线正式切到 best_supervised + flux_guard + no_postcorr，并内置版本修改表	进一步确认这条分支虽然有趣，但不应成为长期默认	形成第一代 notebook 内历史表
v36	回退确认版	回退到已验证最强的 v30 主线，不继续沿用 v33-v35 的低效率分支试探	当前最好主线仍来自 v30 家族	把后续主线重新拉回 v30
v37	历史表成形版	保留 v30 强主线，只比较两个最值得继续追踪的 checkpoint 与一个 no_postcorr 对照，并写入较完整历史表	版本复盘开始被显式纳入 notebook 本体	为 v40-v43 的集成表述提供来源

续下页

版本	阶段定位	主要改动/实验目标	暴露的问题或学到的结论	对当前主线的影响
v38	桥接对照版	在 <code>phase_lockin</code> 家族里增加 <code>soft-postcorr</code> 桥接对照	说明 <code>post-correction</code> 不能只看开/关两极端	只保留为局部结构参考
v39	集成前总结版	延续 v37 的主线与历史表，总结进入集成版前的最强 <code>line-flat</code> 结论	说明下一步该做“集成整理”，而不是继续无限扩 <code>family</code>	直接引向 v40
阶段五：v40-v50，集成 notebook + 历史表出现期				
v40	集成主线版	保留 v30 强主线，并把局部 <code>quality sweep</code> 收回到真正赢出的 <code>baseline + best_hybrid_polish</code> 附近	集成版的目标是收束，而不是继续扩展	成为后续 v44 的直接前身
v41	结构问题先修版	先修参考相位与 ROI 覆盖两个结构问题，再做局部 <code>quality sweep</code>	说明部分表现问题来自结构而非权重	为 v42a/v42b 单变量实验铺路
v42a	第二主拐点	单变量 <code>ablation</code> ：只改 Cell5 参考相位生成的 <code>efficiency_floor=0.05</code> ，其余与 v39 一致	证明 <code>reference-floor</code> 修复是有效、可独立复用的	成为后续集成版必须继承的修复
v42b	ROI-only ablation	只改 <code>line-flat</code> ROI 覆盖，不动 <code>reference floor</code> 或 <code>base cfg</code>	ROI 扩展会改变 <code>overlap / uniformity</code> ，但不足以成为默认主线	只保留为历史 <code>ablation</code> 注释
v43	第三主拐点	把 <code>checkpoint</code> 路径从共享 <code>wider_net</code> 显式拆成 <code>wider_net_v30_anchor</code>	代码版本和权重版本必须分开管理，否则会互相覆盖	成为后续训练/导出/benchmark 的路径管理基础
v44	第四主拐点	集成 v30 骨架、v42a <code>floor</code> 修复、v43 <code>checkpoint</code> 分离和 <code>core-phase-first</code> 报告	形成了一个可以长期迭代的“干净基座”	是 v45-v52 的共同参考底座
v45	展示清理版	保持 v44 主线不变，修正乱码和 <code>best-route</code> 图片摆放	结果页可读性本身会影响版本判断	只影响展示层，不改算法骨架
v46	算法升级版	在 v44 骨架上加入 GPU-only <code>hybrid polish</code> 、 <code>phasor phase prediction</code> 、 <code>output-intensity</code> 、 <code>TV</code> 、 <code>teacher annealing</code> 、 <code>CPU-first dataloading</code>	这是 v44 之后最明确的一次算法增量升级	保留为重要实验分支，但未成为最终稳定交付基座
v47	展示稳定版	继续清理结果查看文本与图片位置，使 <code>Change Log / next-step guidance</code> 更可读	说明文档层面的清晰表达开始和算法同等重要	后续 v48-v49 基本沿用这一版的展示口径
v48	继承漂移版	标题虽为 v48，但首段仍沿用“v47 展示清理”叙述	文案继承开始落后于真实改动	作为版本史重建时的典型漂移样本
v49	继承漂移版	延续 v48 的展示口径，历史表头和“当前版本如何处理”继续滞后	说明单靠首段和历史表已经无法可靠判定真实变化	强化了人工重建版本史的必要性
v50	高分辨率前夜版	基于 v44 做 <code>balanced</code> 权重配置，并修早期编码问题；但标题/说明仍写成 v44	<code>core uniformity</code> 有改善，但 <code>efficiency_floor=0.05</code> 让效率开始塌到约 11%	被 v53 明确判定为已被替代
阶段六：v51-v58，高分辨率 / line-flat / 结构一致性修复期				
v51	文案修复版	修复中文编码（乱码），并把 <code>route</code> 图片移到最后一个结果单元；标题/首段仍沿用 v44	进一步确认低效率根因不是文案，而是 <code>floor</code> 设定	被 v53 判定为过渡版
v52	高分辨率引入版	SLM 分辨率升到 1024x1024，新增 <code>line_flat_top_target</code> ；标题/首段仍沿用 v44	高分辨率和 <code>line-flat</code> 都能跑，但效率仍因旧 <code>floor</code> 设定塌陷	成为 v53 必须修复的直接前件

续下页

版本	阶段定位	主要改动/实验目标	暴露的问题或学到的结论	对当前主线的影响
v53	第五主拐点	把 <code>efficiency_floor</code> 从 0.05 提到 0.80, 并同步提高 <code>efficiency</code> 权重、减轻 <code>core-phase</code> 过拟合倾向	高分辨率 + <code>line-flat</code> 主线重新变得可用, 效率目标回到合理量级	是高分辨率阶段的算法修复锚点
v54	分支延续版	延续 v53 高分辨率分支, 但标题/首段仍沿用 v53	元数据漂移继续存在, 说明实验版本和文案版本脱节	只保留为复跑记录
v55	分支延续版	继续沿 v53 分支复跑, 并暴露更多结构一致性问题; 标题/首段仍沿用 v53	说明下一步问题已从参数转到 <code>notebook</code> 结构与一致性	直接引向 v56 的结构修复
v56	第六主拐点	修结构一致性: 去掉重复写文件循环、 <code>reference</code> 生成改用 <code>cfg.efficiency_floor</code> 、统一 <code>route score</code> 、统一 <code>floor=0.15</code> 、训练 <code>floor=0.30</code> 、 <code>output_size</code> 提到 4096、补 ROI 断言	当前主要风险从“算法方向”转为“结构和一致性”	成为当前稳定交付线的核心基础
v57	结构修复未收口版	基于 v56 继续整理, 但标题仍写成 v56, 且 <code>benchmark</code> 结果写入/读取路径仍不一致, 导出段也有脱节	说明生成器与成品 <code>notebook</code> 已经不同步	作为 v58 的直接修补对象
v58	当前稳定附件	保留 v56 结构修复主线, 补回真实导出单元、统一 <code>benchmark_lin2025_hybrid/summary.json</code> 路径、补 <code>best_route_reason</code> 与版本一致性, 并改成更安全的运行目录	当前它更像结构/一致性稳定版, 而不是新的算法主拐点	现阶段推荐同步进汇报包的稳定 <code>notebook</code>

从全系 `notebook` 看, 真正决定当前主线的不是所有版本, 而是少数关键拐点: v30 确立了可复用的 `learned` 主线骨架, v42a 验证了 `reference-floor` 修复, v43 解决了 `checkpoint` 覆盖问题, v53 把高分辨率 + `line-flat` 从低效率状态拉回可用区间, v56 则把结构一致性和评分逻辑修到可继续迭代的程度。当前推荐同步进汇报包的附件是 v58, 因为它把导出、`summary` 路径和版本一致性进一步收口; 但就“算法主拐点”而言, v58 更像稳定交付版, 而不是新的算法代际。

3 三种方法的原始工作逻辑与当前定位

3.1 795: 实验经验参考路线

原始内容和逻辑 795 方法最初对应的是实验系统中的经验参考相位和工程优化流程。其核心特点不是“统一的现代优化框架”, 而是在特定实验条件下逐步调试得到的一套可用方案。它首先解决的问题不是“如何抽象定义一个通用全息优化器”, 而是“在当前设备、当前目标和当前实验链路下, 怎样稳定地产生一束能真正用于实验的平顶光”。

从 `references/ftl_gen/795.py` 的原始实现来看, 该方法是围绕真实实验分辨率和真实控制链路展开的。脚本中 SLM 面尺寸直接取为 $N=[1024, 1272]$, 而输出面则扩展到更大的计算网格 $NT=[6876, 6876]$; 入射振幅由 SLM_1X 中的激光高斯包络生成, 随后再在输出面构造目标场与加权区域。这里最值得强调的是: 原始 795 并不是简单读取一张已有相位

图，而是显式定义了目标振幅、目标相位、加权掩模、初始猜测相位和若干种不同的 cost function，然后通过多轮优化与实验反馈共同得到最终可用的相位。

原始 795 思路高度依赖实验上下文。相位文件中的 `d`、`dx`、`dy` 等信息直接对应目标宽度、偏移位置和实验视场中的几何关系，而不是仅供命名使用的标签。更重要的是，脚本默认目标并不是一般意义上的圆形平顶，而是通过 `slm.gaussian_line2(...)` 构造的线形平顶光：沿一个方向具有有限平顶宽度，沿另一方向保持高斯衰减。因此，从一开始它就是围绕具体实验目标而设计的。

从原始脚本的执行顺序看，795 的工作流程可以更具体地概括为：

1. 在输出面利用 `slm.gaussian_line2(...)` 构造线形平顶目标振幅 T_a ，并用 `slm.phase_flat(...)` 构造目标相位 P ；
2. 通过 `slm.flat_top_round(...)`、`slm.weighting_value(...)` 等操作构造加权区域 W_{cg} ，并进一步利用目标强度的前 99% 区域形成 W_{99} ，最后令 $W_{cg}=W_{99}$ ，从而把优化重点集中在实验最关心的核心区域；
3. 将目标振幅和目标相位与 W_{cg} 相乘，并按入射光总能量重新归一化，使目标总强度与实际激光功率处于一致量级；
4. 根据 `dx`、`dy` 以及实验几何关系，调用 `phase_guess(...)` 构造初始相位。该初值并不是随机相位，而是显式包含线性倾斜、二次曲率和环形项，用来控制傅里叶平面的偏移、尺寸和包络形状；
5. 在传播模型方面，脚本通过 `SLM_1X` 和自定义 Fourier operator 将 SLM 面场传播到输出面，并在输出面上计算振幅、相位、重叠积分和效率；
6. 在目标函数方面，脚本并非只使用一种代价函数，而是并行尝试了若干类 cost：一类是加权区域内的振幅平方误差，另一类是基于目标与输出内积的 overlap 误差，还有一类是在 overlap 的基础上再叠加 $1/\eta^2$ 、 $1/\eta^4$ 、 $1/\eta^8$ 等效率惩罚项，以便在均匀性和效率之间寻找合适平衡；
7. 在优化流程方面，脚本会多次调用 `CG_2` 和 `CG_new`，包括短步数预优化、双阶段优化以及更长步数的精修，并把阶段性结果保存为 `phaseold_wcg=... .npy` 一类文件；
8. 在真实实验链路中，脚本进一步接入 `IDS_Camera.GetImage()` 获取实测图像，裁剪兴趣区域、计算质心、修正目标振幅，再重新运行共轭梯度优化，并通过 `slm_play.update(...)` 把更新后的相位重新送回 SLM，形成“测量-修正-再优化”的闭环。

从方法论上看，795 更像“实验目标构造 + 加权区域设计 + 共轭梯度优化 + 实测反馈修正”的路线，而不是“先训练一个模型，再统一泛化到所有目标”的路线。其优势在于，它直接面对真实装置，因此很多理论模型中未显式写出的误差，例如 SLM 响应不理想、光束椭圆、像差和装调偏差，都可能通过长期调试被隐式吸收进目标定义、权重设计和参考相位之中。

参考价值 795 的参考价值首先在于它保留了最贴近真实实验的经验先验。它说明，在特定系统下，怎样的目标构造、怎样的初始相位和怎样的误差修正链路是真正有效的。对当前项目而言，这种价值并不体现在“它一定是当前指标下的最优解”，而体现在它告诉我们：原始系统里哪些结构是稳定可用的，哪些相位先验值得保留。

同时，它也提供了一个非常有意义的历史基线。只要把 795 适配到当前仿真框架，我们就可以回答一个关键问题：新程序相对于原始经验方法究竟提升了多少。它的局限同样值得强调：其一，目标形状与当前默认圆形平顶并不完全一致；其二，它工作在真实实验分辨率和真实控制链路上，而当前训练更多是低分辨率仿真；其三，它对参数变化较敏感，泛化能力有限。

新程序如何借鉴 当前新程序并没有抛弃 795，而是把它从“最终执行方法”提升成“参考相位先验、teacher 来源和对照基线”。也就是说，795 在当前系统中的作用主要有三类：

1. 作为 **参考相位先验**，为当前系统提供可解释、可落地的初始化来源；
2. 作为 **teacher 来源之一**，参与 Lin2025 训练；
3. 作为 **benchmark 基线**，用于判断新系统到底提升了多少。

因此，当前新程序对 795 的借鉴不是复制整条旧实验链路，而是提取其中最有价值的部分，即参考相位、目标构造经验和实验先验，并把这些内容迁移进一个更统一、更可比、也更容易继续迭代的框架之中。

3.2 Bowman/hybrid: 物理优化主线

原始内容和逻辑 Bowman 方法的原始思路来自对全息问题的物理逆问题建模。与经验参考路线不同，它并不预设一个已经调好的参考相位，而是把 SLM 相位本身作为主要优化变量。对于给定的入射场、给定的传播模型和给定的目标场，算法通过不断修改相位，使传播后的输出场尽量逼近期望目标。

从 Bowman 原论文可知，其核心逻辑可以概括为：

1. 设定 SLM 面入射场 $E_{p,q}^{\text{in}} = S_{p,q} \exp(i\phi_{p,q})$ ，其中 $S_{p,q}$ 是固定入射振幅， $\phi_{p,q}$ 是待优化相位；
2. 通过传播子把 SLM 面场映射到输出面，在远场近似下可写为快速傅里叶变换；
3. 在输出面定义目标场，并在 measure region 内构造 cost function；
4. 该 cost function 不是简单只比较强度，而是通过目标场与输出场的内积来定义 fidelity，从而同时约束幅度和相位；
5. 采用包含线性倾斜和二次曲率项的初始 guess phase，用来控制输出包络的位置与尺度，并抑制涡旋导致的停滞；

6. 使用 conjugate gradient minimisation 沿共轭方向更新相位，直到 cost function 收敛或停滞。

从原论文的观点看，Bowman 方法最关键的优势在于它把“目标场逼近”写成了一个物理上清晰且数学上可微的优化问题，因此每一步都可以被解释为：当前输出与目标偏差有多大，应该沿哪个方向改相位才能更快减小误差。与传统 IFTA 相比，它更强调对 cost function 的精心设计，以及通过合适的初始相位避免光学涡旋和早期停滞。

在当前工程里，Bowman 思路被改造成了 hybrid 主线。虽然实现形式已经不再是最原始的论文版本，但核心精神仍然保留：直接在物理传播模型下优化相位，并让结果在测量区域内尽量逼近目标场。

参考价值 Bowman/hybrid 的参考价值主要在于，它提供了当前系统中最可靠的物理收尾主线。它的优势不是“计算一定最快”，而是“物理一致性最强、结果最可解释、后端收敛最稳定”。只要初值不完全偏离可行区域，它通常都能把结果拉回一个较合理的解。

同时，它也暴露出当前问题的重要事实：该问题是明显的多目标非凸问题。核心区均匀性、核心区等相位和效率之间存在 trade-off，不同初始化会把优化导向不同局部区域。因此，Bowman/hybrid 既是强基线，也是判断初始化质量是否真正有价值的重要标准。

新程序如何借鉴 当前新程序并没有试图用神经网络替代 Bowman。恰恰相反，Bowman/hybrid 被保留为系统最核心的后端模块。新程序对它的借鉴主要体现在两方面：

1. 保留“在物理传播模型下直接优化相位”的主思想；
2. 在工程上加入多分辨率 refine、多阶段 polish、warm start 和多候选初始化，使其更适合当前平顶光任务。

因此，当前系统真正做的事情，是通过引入学习型初始化，把 Bowman/hybrid 从“强但依赖初值的优化器”转变成“在更优初值支持下可以到达更好 Pareto 点的后端主线”。从汇报角度看，这一点尤其重要：当前方法的成功并非来自单纯引入神经网络，而是来自“学习型初始化 + 物理一致收尾”的结合。

3.3 Lin2025：学习型初始化路线

原始内容和逻辑 Lin 原论文的原始目标并不是“为平顶光直接生成实验相位”，而是“为大规模中性原子阵列重排实时生成 hologram”。因此，Lin 方法的原始工作逻辑并不是单独的一个神经网络，而是一个完整系统的一部分。根据原文，其核心流程可以概括为：

1. 先利用 WGS 算法为初始和目标 tweezer 阵列生成 hologram，形成训练样本；
2. 使用 EMCCD 采集原子占据图像，并通过一个三层卷积网络识别每个 trap 位点是否存在单原子；

3. 使用 Hungarian algorithm 并结合 block decomposition 计算原子与目标位点的最优匹配路径，使路径规划在大规模阵列下仍接近常数时间；
4. 将整个移动过程分解为若干小步，每一步只允许较小的位移和相位变化，以减少加热和损失；
5. 用 fully convolutional neural network 计算每一步所需的 hologram；
6. 由于 hologram 属于频域量，网络并不是直接输出频域像素，而是先在位置域预测振幅和相位，再通过 inverse FFT 得到最终 hologram。

Lin 原文中最值得注意的思想有三点。第一，AI 模型并不是孤立存在的，而是嵌入在“图像识别、路径匹配、小步搬运、实时 hologram 生成”的整个系统里。第二，网络结构不是简单粗暴地直接预测频域相位，而是利用“卷积网络更擅长处理位置域结构、频域通过 FFT 获得”的事实。第三，Lin 方法追求的是实时性和可扩展性，即在原子规模增加时仍尽可能保持近似常数时间的计算开销。

在当前项目中，Lin2025 已经被重构为面向平顶光初始化学习的版本。网络输入包括目标振幅、目标相位、核心区 mask 和 ROI mask；输出先在位置域预测，再通过 FFT 转成 hologram 初值。由此可见，当前版本保留的是 Lin 的“学习型 hologram 生成”思想，而不是整套原子重排系统。

参考价值 Lin 方法的参考价值主要在于它提供了一种与经验法和纯物理法都不同的思路：通过训练，让网络学习“什么样的初始化更可能把后端优化带入高质量区域”。这一点对当前项目尤其重要，因为当前问题的瓶颈往往不只在后端优化器是否足够强，还在于初始化是否足够好。

同时，Lin 原文还提供了非常有启发性的结构设计经验，例如位置域预测再经 FFT 得到频域 hologram、使用卷积与残差结构建模空间信息、以及把模型放入完整流程而不是孤立使用。当前项目即便没有复现原文中的原子占据识别和路径规划，也仍然能从这些设计中获益。

Lin 方法的局限也需要说明。原文服务的是原子阵列重排问题，而不是当前平顶光问题，因此它不能被直接照搬。当前项目若要真正吸收 Lin 的优势，必须结合自身目标重新定义输入、损失和评价标准。

新程序如何借鉴 当前新程序对 Lin 的借鉴，是有选择地吸收其中与 hologram 学习最相关的部分，而不是复现整套原子重排系统。具体而言，新程序主要借鉴了：

1. 位置域输入、位置域预测、再经 FFT 得到 hologram 的思路；
2. 卷积主干配合残差结构的网络形态；
3. 把网络作为初始化器，而不是直接替代物理优化器。

因此，当前 Lin2025 更准确的定位应当是：它是受 Lin 原文启发而重构出的学习型全息初始化模块，而不是原论文工作流程的直接复现。与此同时，它也是当前系统中最适合持续迭代的部分。无论未来要做更高分辨率训练、强化 teacher 策略，还是把 camera-in-the-loop 接入训练链路，最自然的扩展入口都在 Lin2025 这条线上。

4 三种方法的关键差异

4.1 目标与使用方式的差异

方法	主要目标	当前工程中的作用	局限
795	提供实验可用相位和实验参考	参考相位、teacher 来源、实验基线	泛化能力有限，对新目标和新分辨率不够稳健
Bowman/hybrid	在物理模型下直接优化最终 SLM 相位	强基线、最终物理收尾	受初值影响较大，单独使用时搜索效率有限
Lin2025	学习较优初始化分布	学习型初始化器，与 hybrid 联合使用	单独输出质量不足，不能稳定替代物理收尾

4.2 分辨率与目标形状差异

在原始实验链路中，795.py 工作在真实 SLM 分辨率和真实实验目标上；而当前 AutoDL notebook 主线（尤其 v40-v58，当前稳定附件为 v58）和训练流程主要面向低分辨率仿真验证。这带来两个直接差异：

1. **分辨率差异：**当前训练和 benchmark 通常工作在 256x256 级别，真实实验则对应更高的 SLM 分辨率。因而当前 AutoDL notebook 主线生成的相位不能视为“直接可上机”的最终实验相位，而更接近算法验证和初始化学习结果。
2. **目标形状差异：**当前默认 flat_top_target 更偏向径向对称的圆形平顶，而原始 795.py 的目标更接近线形平顶光。因此，两者在目标定义上并非完全等价。

这意味着：当前系统与原始 795 方法并不是简单替代关系，而是“在原有参考思路基础上面向仿真训练与后续扩展的重构版本”。

4.3 相机反馈差异

原始 795 实验链路通常包含实测反馈，即根据相机采集的输出对目标或相位进行修正。当前系统虽然在代码中已经具备相机闭环接口，但当前 AutoDL notebook 默认主流程并未真正接入相机，仍以仿真闭环为主。

因此在汇报时应准确表述为：

当前系统已经完成了仿真训练、仿真初始化和物理后端优化的整合，但尚未完全恢复到原始 795 实验流程那种“实时相机反馈”的闭环状态。

5 神经网络与传统方法的互补关系

5.1 为什么不能只保留一种方法

从当前结果看，三种方法都不能单独覆盖全部需求：

1. 只用 795，参考价值高，但在当前统一指标下不够强；
2. 只用 Bowman/hybrid，物理可解释性强，但对初值敏感；
3. 只用 Lin2025，网络输出作为最终相位不稳定，lin2025_only 指标明显偏弱。

因此，当前系统采用混合路线并不是“复杂化设计”，而是因为该问题天然同时需要：

1. 实验先验；
2. 可学习的初始化；
3. 物理一致的后端收敛。

5.2 当前最合理的关系描述

对三种方法的关系，可以用如下形式概括：

795 提供参考先验 → Lin2025 提供学习型初始化 → Bowman/hybrid 完成物理收尾

其中：

1. 795 解决“经验与实验参考”的问题；
2. Lin2025 解决“更好初值”的问题；
3. Bowman/hybrid 解决“最终质量收敛”的问题。

6 相对于 795 的改进点

6.1 从单参考相位变为多候选初始化

原始 795 更接近固定参考相位思路，而当前系统在 `runner.py` 中已经形成多候选初始化机制，可以比较：

1. `reference`;
2. `lin2025`;
3. `warm_start`;
4. 以及它们的混合版本。

这意味着系统不再依赖一张固定参考相位，而是会围绕当前目标与历史信息，自动选择更合适的起点。

6.2 从经验法扩展为可训练框架

与原始 795 相比，当前系统的一个关键改进是引入了可训练网络。网络通过 teacher 蒸馏、监督损失和质量损失共同学习，使得系统可以通过迭代训练逐步改善初始化质量，而不只是手工调经验参数。

6.3 从偏效率优化扩展为三指标统一分析

原始 795 更偏工程可用与效率导向，而当前系统明确使用以下三项指标进行统一分析：

1. 核心区均匀性；
2. 核心区等相位；
3. 光利用效率。

这一点使得系统可以更清楚地区分“均匀性更好但效率下降”和“效率更高但相位较差”的不同类型解。

7 相对于 Bowman/hybrid 的改进点

7.1 更强的初始化能力

Bowman/hybrid 单独使用时，质量很大程度受初始化影响。Lin2025 的加入并没有替代 Bowman，而是显著提升了初始化质量，使得后续物理优化更容易进入较优区域。

7.2 可以到达新的 Pareto 点

在当前结果中，lin2025_plus_hybrid_balanced 或 lin2025_plus_hybrid_quality 往往能够在“核心区均匀性、核心区等相位、效率”的多目标空间中，找到与 hybrid_only 不同的折中点。也就是说，神经网络的引入并不一定让三项都绝对更优，但可以让系统到达纯 Bowman 路线较难到达的新 Pareto 区域。

7.3 仍然无法完全替代 Bowman

从 lin2025_only 的结果可以看出，当前网络直接输出的相位尚不能稳定达到最终实验要求。因此，Lin2025 与 Bowman/hybrid 的关系仍然是“前端补强”，而不是“后端替代”。

8 当前结果说明了什么

8.1 当前推荐 notebook 与文档映射

本汇报包当前同步的远程训练附件为：

```
notebooks/train_lin2025_autodl_lineflat_highres_export_v58.ipynb
```

这一定义需要和版本史一起理解。第一，本文档对“当前 AutoDL notebook 主线”的描述主要依据 v40-v58 这条集成版演进线，而不是仅根据单个 v58 文件。第二，当前真正决定算法方向的关键拐点是 v30 / v42a / v43 / v53 / v56；其中 v58 的主要价值是把导出、summary 路径、版本标注和运行目录等结构一致性问题进一步修平，使其更适合作为汇报包附件。第三，因此本文凡是提到“当前稳定 notebook”，都应理解为：

算法主线来自 v40-v58 的集成演进，当前可交付附件取 v58。

8.2 核心结果对比表

为便于汇报时直接比较不同路线的实际效果，表 2 汇总了当前几类最有代表性的结果。表中所列指标均来自项目当前已保存的 benchmark 或 checkpoint 分析结果，其中：

- 1. 795_only 表示适配到当前仿真框架后的参考基线；
- 2. hybrid_only 表示 Bowman/hybrid 强基线；
- 3. balanced 和 quality 表示当前混合系统的两种主要运行模式；
- 4. AutoDL best supervised quality 表示远程训练后经本地分析得到的一组更强 checkpoint 结果。

表 2: 当前代表性路线的结果对比

路线	核心区均匀性 损失	核心区相位 平坦度	效率
795_only	0.02897	0.02716	0.25359
hybrid_only	0.03093	0.00302	0.85184
lin2025_plus_hybrid_balanced	0.03003	0.00192	0.82033
lin2025_plus_hybrid_quality	0.02693	0.00408	0.79291
AutoDL best supervised quality	0.02640	0.00134	0.78949

从表 2 可以直接看到三点。第一，相对于 795_only，当前系统无论在核心区等相位还是效率上都已有显著提升，因此 795 更适合作为参考基线，而不是最终主路线。第二，相

对于 `hybrid_only`，当前系统并没有形成单一意义上的“全面超越”，而是在不同路线下给出不同类型的改进：`balanced` 更强调在前两项上同时优于 Bowman 基线，`quality` 更偏向进一步压低核心区均匀性，而 AutoDL 训练得到的更优 `checkpoint` 则证明了系统在前两项上还有进一步提升的潜力。第三，效率仍然是当前最容易受到牺牲的一项指标，因此后续实验若真正投入装置，应保留多条路线作为备选，而不宜只保留单一路线。

8.3 对 795 的提升

当前系统相对于 `795_only` 的提升已经较为明显，主要体现在：

1. 核心区等相位显著改善；
2. 效率大幅提高；
3. 多数情况下核心区均匀性也有改善。

因此，795 在当前项目中更适合作为参考基线和 `teacher` 来源，而不再是最终主路线。

8.4 对 Bowman 的提升

当前系统相对 `hybrid_only` 的提升是“有条件成立”的：

1. 在某些结果中，`quality` 路线可以进一步压低核心区均匀性；
2. 在某些结果中，`balanced` 路线能够在核心区均匀性和核心区等相位上同时优于 `hybrid_only`；
3. 但效率通常仍会有所下降。

因此当前更准确的表述不是“已经完全超过 Bowman”，而是：

当前系统已经在若干关键指标上找到了优于 `hybrid_only` 的解，但尚未在三项指标上同时稳定全面超越 Bowman 风格基线。

9 潜在问题与风险

9.1 当前最主要的潜在问题

当前系统仍然存在以下问题：

1. 分辨率尚未直接对齐实验系统；
2. 默认目标仍偏圆形平顶，而非线形平顶；
3. 缺少真正接入相机的默认闭环流程；
4. `checkpoint` 结果存在波动，说明 `teacher` 策略与多目标平衡仍较敏感；
5. 效率仍可能在质量优先模式下下降，实验上需要保留 `fallback` 路线。

9.2 为什么实验中效率可能反而下降

这是当前系统中必须明确说明的一个现象。原因在于：

1. 当前优化目标更强调核心区均匀性和核心区等相位；
2. 为了把核心区修平，系统会重新分配部分能量；
3. 因而效率可能不如纯 `hybrid_only` 或原始实验经验解。

因此，如果真正投入实验，应同时保留三条路线：

1. `hybrid_only`：效率保底；
2. `balanced`：前两项与效率的折中；
3. `quality`：核心区质量优先。

10 后续最值得优化的方向

结合当前结果，后续优化最值得投入的方向有四项：

1. 把目标定义进一步向真实实验目标对齐，特别是线形平顶目标；
2. 把分辨率与真实 SLM 对齐，减少从低分辨率算法验证到高分辨率实验加载之间的误差；
3. 完善 `camera-in-the-loop`，把相机反馈真正接入默认主流程；
4. 稳定 `teacher` 策略与 `checkpoint` 选优规则，降低训练波动。

11 结论

三种参考方法在当前项目中并不是相互替代关系，而是分工不同、相互补充的三类技术来源。795 提供实验经验和参考先验，Bowman/hybrid 提供物理一致的最终优化，而 Lin2025 提供可学习的高质量初始化。当前系统相对于 795 已经实现了明显提升，相对于 Bowman/hybrid 也已经能够在若干关键指标上找到更优解，但仍未实现三项指标的全面稳定统一最优。总体而言，现阶段最合理的定位是：当前系统已经从原始 795 经验方法发展为一个可训练、可比较、可扩展的混合优化框架，并具备进一步向真实实验闭环推进的基础。